

CİNESEAT · DİREKTÖR SUNUMU

Frontend–Backend Entegrasyonu Sunum Akışı

Backend mimarisinin tamamlanmasının ardından kurulan frontend–backend bağlantısının yönetim sunumu için hazırlanmış akış — konuşma notları, kod örnekleri ve rakamlarla.

Sunum süresi (önerilen): 12–15 dakika + soru-cevap

Hedef kitle: Direktör / teknik olmayan-orta düzey teknik yönetim

Eşlik eden doküman: CineSeat_Frontend_Backend_Entegrasyon_Raporu.pdf (derinlemesine teknik referans)

Tarih: 28.08.2026

Bugün Ne Anlatacağız?

Backend mimarisi (Clean Architecture + CQRS) tamamlandı. Bu sunum, o backend'in **React tabanlı frontend'e nasıl bağlandığını** — hangi teknik kararların alındığını, bu kararların neden önemli olduğunu ve bağlantının gerçekten çalıştığının nasıl doğrulandığını anlatıyor.

Problem

→

Mimari

→

3 Temel Sözleşme

→

Kritik Akış Örneği

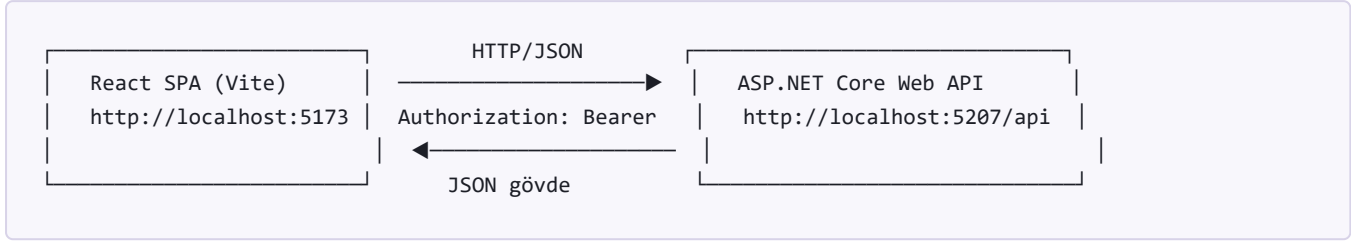
→

Kanıt & Rakamlar

KONUŞMA NOTU

Buradan başla: "Backend ve frontend iki farklı ekip/katman gibi düşünülebilir ama aslında iki ayrı çalıştırılabilir uygulama — ayrı süreç, ayrı port, hatta ayrı sunucuda çalışabilirler. Bugün onları birbirine bağlayan üç temel sözleşmeyi anlatacağım: güvenlik, kimlik ve veri."

İki Ayır Uygulama, Nasıl Konuşacak?



Frontend (kullanıcının tarayıcısında çalışan React uygulaması) ve backend (sunucuda çalışan API), **ayrı süreçler** olarak çalışır ve birbirleriyle yalnızca ağ üzerinden, HTTP istekleriyle konuşur. Bu basit görünen kurulum üç somut mühendislik problemini beraberinde getirir:

1. **Tarayıcı güvenliği:** Farklı port = farklı "origin". Tarayıcı varsayılan olarak bu isteği engeller.
2. **Kimlik doğrulama:** Backend, isteği atan kişinin kim olduğunu her seferinde nasıl bilecek?
3. **Veri sözleşmesi:** İki taraf, aynı veriyi aynı şekilde mi yorumluyor?

KONUŞMA NOTU

"Bu üç problem, her modern web uygulamasında karşımıza çıkar — CineSeat'e özgü değil. Ama nasıl çözüldükleri, sistemin güvenliğini ve bakımını doğrudan belirliyor. Şimdi üçünü sırayla göstereceğim."

Kuş Bakışı Mimari

Backend (Onion / Clean Architecture + CQRS)

Domain → iş kuralları, hiç bağımlılık
Application → Command/Query + Handler
Infrastructure → JWT, parola hash
Persistence → EF Core + PostgreSQL

Frontend (React)

pages/ → ekranlar
components/ → yeniden kullanılan UI
hooks/ → paylaşılan davranış
services/ ★ → backend'e giden HER istek

Backend, sorumlulukları katmanlara ayıran **Onion (Clean) Architecture** ile; frontend ise her isteğin tek bir noktadan (`services/`) geçtiği bir servis katmanı ile kurulmuştur. Backend tarafında her iş eylemi (giriş yapma, koltuk kilitleme, rezervasyon oluşturma) **CQRS** deseniyle bir Command/Query + Handler ikilisine karşılık gelir.

Neden önemli? Bu ayırım sayesinde iki taraf birbirinden **bağımsız olarak** geliştirilip test edilebiliyor — backend ekibi veritabanı şemasını değiştirirken frontend'in çalışmayı durdurumaması, ya da tam tersi, sözleşmenin (aşağıda anlatılacak) korunmasına bağlı.

CORS — Kim Kiminle Konuşabilir?

CORS (Cross-Origin Resource Sharing), backend'in "hangi adresten gelen istekleri kabul ediyorum" sorusuna verdiği açık cevaptır. Tanımlanmazsa tarayıcı isteği kendiliğinden engeller — kod doğru olsa bile.

```
backend/Presentation/CineSeat.WebAPI/Program.cs
```

```
policy.WithOrigins("http://localhost:5173", "http://127.0.0.1:5173")
    .AllowAnyHeader()
    .AllowAnyMethod();
```

İnce ama kritik detay: Bu satır, kimlik doğrulamadan (`UseAuthentication`) **önce** çalıştırılmalı. Tarayıcı asıl isteği göndermeden önce token'sız bir "ön-uçuş" isteği (`OPTIONS`) atar; sıra yanlış olsaydı bu ön-uçuş isteği reddedilir, hata mesajı da yanıltıcı bir "CORS hatası" gibi görünürdü. Küçük bir sıralama kararı, üretimde saatler süren yanlış teşhise yol açabilecek türden bir detay.

KONUŞMA NOTU

"CORS'u bir güvenlik duvarı gibi düşünün: yalnızca bizim frontend'imizin adresine izin veriyoruz, başka hiçbir web sitesi bu API'yi tarayıcı üzerinden çağıramaz."

JWT — Kimlik Doğrulama

- 1) Giriş POST /api/auth/login { usernameOrEmail, password }
- 2) Yanıt { token, expiresAt, user }
- 3) Saklama sessionStorage["cineseat_user"] = { ...user, token }
- 4) Her istek Header: Authorization: Bearer <token>
- 5) Doğrulama backend imzayı kontrol eder → izin ver / 401 döndür

Sistem **durum tutmayan (stateless) JWT** kullanır: backend, oturumu kendi hafızasında saklamaz. Token'ın kendisi — imzalı, sahtesi yapılamaz bir şekilde — kullanıcının kimliğini, rolünü ve izinlerini taşır.

backend — JwtTokenService.cs

```
expires: DateTime.UtcNow.AddMinutes(_settings.ExpiryMinutes),  
signingCredentials: new SigningCredentials(key, SecurityAlgorithms.HmacSha256)
```

Güvenlik güvencesi: Uygulama açılışında imza anahtarının en az 32 karakter olduğu denetlenir — kısa bir anahtar HS256 imzasını kırılabilir kılar. Bu kontrol geçmezse **uygulama hiç ayağa kalkmaz**; zayıf bir yapılandırmayla sessizce çalışmaya devam etmez.

KONUŞMA NOTU

“Token, bir tiyatro bileti gibi düşünülebilir: içeriği (kim, hangi yetkiyle) üzerinde yazılı ve sahtesi kolayca anlaşılır. Sunucu her seferinde 'bu kişi kim?' diye veritabanına sormak zorunda değil.”

Veri Sözleşmesi — DTO Eşleme

Backend'in gönderdiği veri şekli (DTO) ile frontend'in ihtiyaç duyduğu şekil her zaman birebir aynı olmayabilir. Her frontend servis dosyası, gelen veriyi kendi bileşenlerinin anlayacağı forma çeviren küçük bir "çevirmen" fonksiyon içerir.

frontend – showtimeService.js

```
function mapShowtimeDto(dto) {
  return {
    id: dto.id, movieId: dto.movieId,
    basePrice: Number(dto.basePrice) || 0,
    format: dto.format,
    hallName: dto.hallName ?? "",
  };
}
```

İş değeri: Backend bir alanı yeniden adlandırırsa ya da tipini değiştirse bile, bu değişikliğin etkisi **tek bir dosyaya** hapsedilir. Bu, iki ekibin birbirini beklemeden, birbirini kırmadan paralel çalışabilmesinin mimari garantisidir — entegrasyon hataları erken ve tek noktada yakalanır.

KONUŞMA NOTU

"Bunu bir çeviri katmanı gibi düşünün: backend kendi diliyle konuşur, frontend kendi diliyle — aradaki 20 küçük 'çevirmen' dosya bu iki dilin hep anlaşmasını sağlıyor."

Bir İsteğin Tam Yolculuğu: Giriş (Login)

Soyut kavramları somutlaştırmak için tek bir isteği baştan sona takip edelim — bu, projedeki **her** yazma işleminin (kayıt, rezervasyon, yorum, kampanya) izlediği aynı iskelettir.

Controller → MediatR → ValidationBehavior → Handler → DTO → JSON Yanıt
(HTTP'yi (isteği doğru (kurallar (iş mantığı, (temiz
devreder) yere yönlendirir) geçerli mi?) veritabanı) veri şekli)

AuthController.cs

```
[HttpPost("login")]  
public async Task<IActionResult> Login([FromBody] LoginCommand command)  
    => Ok(await _mediator.Send(command));
```

KONUŞMA NOTU

"Controller'ın kendisi hiçbir iş mantığı içermiyor — sadece isteği doğru yere yönlendiriyor. Bu, kodun test edilebilirliğini ve okunabilirliğini doğrudan artırıyor."

Güvenlik: Kasıtlı Belirsiz Hata Mesajı

LoginCommandHandler.cs

```
// Kullanıcı yok İLE parola yanlış AYNI mesajı döndürür: farklı mesaj  
// vermek saldırgana hangi e-postaların kayıtlı olduğunu sızdırır.  
if (candidate is null || !_passwordHasher.Verify(...))  
    throw new UnauthorizedException("Kullanıcı adı/e-posta veya parola hatalı.");
```

Bu tek satır, OWASP'ın kimlik doğrulama en iyi pratiklerinden biri olan **user enumeration önleme**'yi uyguluyor: “bu e-posta kayıtlı değil” ile “parolanız yanlış” arasındaki fark, saldırganlara sistemde kayıtlı e-posta adreslerini tek tek denemeleri için bir araç verirdi.

KONUŞMA NOTU

“Bu, entegrasyonun sadece ‘çalışması’ değil, ‘güvenli çalışması’ için alınmış bilinçli bir kararın örneği — küçük görünen bir detay ama gerçek dünyada sık istismar edilen bir açığı kapatıyor.”

İki Kişi Aynı Koltuğu Aynı Anda Seçerse?

Bilet satışında en kritik teknik risk budur. Çözüm: veritabanında (Seans, Koltuk) çifti üzerinde bir **benzersizlik kısıtı** ve bunu yöneten bir "devralma" mantığı.

LockSeatCommandHandler.cs

```
var isExpired = existingLock.LockExpiresAt < now;  
var isSameUser = existingLock.UserId == userId;  
  
if (!isExpired && !isSameUser)  
    throw new ConflictException("Bu koltuk şu anda başka bir kullanıcı tarafından kilitli.");  
  
// Süresi dolmuş ya da aynı kullanıcı → satırı devral / yenile.
```

Nasıl çalışır: Veritabanı iki eşzamanlı kilitleme isteğinden birini otomatik olarak reddeder (benzersizlik kısıtı). Kod bu reddi bir hata olarak kullanıcıya yansıtmaz — satırı yeniden okur, süresi dolmuş mu diye bakar, dolmuşsa devralır. Bu deseni dağıtık sistemlerde "iyimser eşzamanlılık kontrolü" olarak adlandırıyoruz.

KONUŞMA NOTU

"Bu, e-ticaretin klasik 'stokta tek ürün kaldı, iki kişi aynı anda satın almaya çalıştı' problemiyle birebir aynı. Burada koltuklar için çözülmüş durumda ve canlı olarak test edildi."

Tek Noktadan Hata Yönetimi

Backend: TEK try/catch (ExceptionHandlerMiddleware)
ValidationException → 400 NotFoundException → 404
UnauthorizedException → 401 ConflictException → 409

Frontend: TEK çeviri noktası (apiClient.js → toApiError)
400/422 → ValidationError 404 → NotFoundError
401 → ApiError 409 → ConflictError

Backend'de hiçbir controller veya iş mantığı kodu kendi içinde hata yakalamaz; hepsi tek bir merkezi middleware'e akar. Frontend tarafında da bu tutarlı hata gövdesi, tek bir fonksiyonda tipli hata sınıflarına çevrilir. Sonuç: **her ekranda tutarlı, öngörülebilir hata davranışı.**

KONUŞMA NOTU

"Bu, 'her sayfa kendi hata mesajını kendi icat ediyor' kaosunu önüyor — bakım ve yeni geliştirici onboarding'i açısından doğrudan zaman kazandırıyor."

Yakalanan ve Çözülen Bir Entegrasyon Hatası

Önce	Sonra
Yönetim paneli <code>pageSize=200</code> gönderiyordu	Backend'in sınırı (100) sayfalar dolaşarak aşıyor
Backend bunu 400 ile reddediyordu	İstek her zaman geçerli sınırlar içinde kalıyor
Ekran "Seanslar yükleniyor..."da donuyordu	Seans listesi sorunsuz yükleniyor

Neden bu vaka önemli: Sorun ne backend'de ne frontend'de tek başına — iki tarafın **aynı sözleşmeye** (sayfa başına en fazla 100 kayıt) sahip olmaması entegrasyon noktasında ortaya çıktı. Bu tür hatalar yalnızca iki tarafı birlikte test ederek yakalanabilir; bu proje bu şekilde test edildi ve düzeltildi.

KONUŞMA NOTU

"Bu, entegrasyon testinin neden ayrı bir aşama olarak önemli olduğunun somut kanıtı — birim testleri iki tarafı da ayrı ayrı 'yeşil' gösterebilirdi, hata yalnızca ikisi birlikte konuştuğunda ortaya çıktı."

Entegrasyonun Kapsamı

20

Frontend servis dosyası

3

Temel sözleşme (CORS / JWT / DTO)

4

Katmanlı backend mimarisi

604Frontend otomatik test (tamamı
geçiyor)**100**

Sayfa başına azami kayıt sınırı

5/5

Koltuk kilidi canlı senaryo testi

KONUŞMA NOTU

"Bu rakamlar, entegrasyonun tek seferlik bir 'bağlantı kuruldu, çalışıyor' işi olmadığını, sürekli test edilen ve büyüyen bir yüzey olduğunu gösteriyor."

Canlı Doğrulama — Sadece Kod Okuması Değil

Bu sunumun arkasındaki iddialar, backend gerçek bir PostgreSQL veritabanına bağlı şekilde ayağa kaldırılarak **canlı olarak** test edildi:

Test	Sonuç
Kayıt → Giriş → Profil → Favoriler uçtan uca akışı	4/4 istek başarılı
Sayfalama sınırı ihlali (3 farklı uçta)	3/3 doğru şekilde reddedildi
Koltuk kilidi: kilitle→devral→yenile→bırak→tekrar bırak	5/5 beklenen sonuç
Yönetici yetkilendirmesi	Doğru çalışıyor

KONUŞMA NOTU

"İddia etmek başka, göstermek başka — bu sunumdaki her teknik iddia gerçek bir çalıştırma ile doğrulandı, varsayımla değil."

Üç Cümlede Entegrasyon

1. **Güvenlik:** CORS, yalnızca bizim frontend'imizin backend'e istek atabilmesini garanti ediyor.
2. **Kimlik:** Stateless JWT, oturumu sunucu hafızasında değil, imzalı bir token'da taşıyor.
3. **Veri:** Her serviste tekrar eden bir "çevirmen" deseni, iki tarafın birbirini kırmadan bağımsız evrilmesini sağlıyor.

Ve en kritik iş akışında — koltuk kilitleme — eşzamanlılık sorunu, veritabanı seviyesinde bir kısıt ve bilinçli bir devralma mantığıyla çözülmüş, canlı olarak test edilmiş durumda.

KONUŞMA NOTU

"Sonuç: bu sadece 'iki uygulama birbiriyle konuşuyor' değil, güvenli, öngörülebilir ve test edilmiş bir konuşma."

Olası Sorular ve Hazır Cevaplar

S: Backend çökerse frontend ne olur?

C: apiClient.js ağ hatasını yakalayıp kullanıcıya "Sunucuya ulaşamıyor" gibi anlaşılır bir mesaj gösterir — teknik bir hata ekranı değil.

S: Token çalınırsa ne olur?

C: Token süreli (dakikalarla sınırlı); süre dolunca backend otomatik reddeder ve frontend oturumu düşürür. Uzun ömürlü kalıcı bir anahtar değildir.

S: Fiyat/indirim hesabını kullanıcı tarayıcıdan değiştirebilir mi?

C: Hayır — istemci yalnızca bir ön izleme gösterir, bağlayıcı hesap her zaman sunucuda, rezervasyon oluşturulurken yeniden yapılır.

S: Bu mimari yeni bir ekrana/uç noktaya ne kadar hızlı uyum sağlar?

C: Yeni bir backend uç noktası için: Command/Query + Handler + Validator (yerleşik desen). Frontend'de: bir servis fonksiyonu + bir `mapXDto`. İkisi de mevcut şablonu takip eder, sıfırdan tasarım gerektirmez.

Teşekkürler

Bu sunumun teknik derinlik gerektiren her iddiası, eşlik eden **“Frontend–Backend Entegrasyonu Teknik Raporu”** dokümanında dosya yolu, sınıf adı ve tam kod parçasıyla belgelenmiştir; sorular için doğrudan referans alınabilir.

KONUŞMA NOTU

“Detaylı teknik sorular için elimizde tam bir yazılı referans var — şimdi sorularınızı alabilirim.”